Volta Industrial: never be the supplier that stops the OEM line

Live predictive maintenance for aluminum EV battery enclosures, on one Databricks platform

For **Alex**, executive sponsor, and **Shreya**, who would own and run the solution

20 minutes · all companies, people and data are fictional and synthetic

6:12 on a Tuesday, Plant East

0:00 to 1:00 · the hook

> The frame welder on the Aurora Elbil line has been running hot for hours. Nobody sees it. At 6:12 it trips. Bjørn, the maintenance supervisor, gets the call. The repair takes **112 minutes**. The JIS buffer to Aurora's assembly line lasts **60**.
> Aurora's line stands still for 52 minutes. The contract charge is about **$575,000**. And Volta gets a mark on the scorecard that decides the next vehicle program.

What we found in your maintenance history: 4 out of 5 repairs are emergencies like this one. Every unplanned repair (112 min) is longer than every buffer you have (45 to 90 min).

THE QUESTION What if Bjørn had known **four hours earlier**, and fixed it in a planned 28-minute break?

Why keep listening: the value, and today's roadmap

1:00 to 3:00 · value first, then the roadmap · illustrative, your finance team replaces the assumptions in the pilot

Own P&L

about $3.6M / yr

190 hours less line downtime, fewer expedited parts

OEM charges avoided

about $4.0M / yr

if 5% of failures reach the OEM (2%: $1.6M; 10%: $8.0M)

Warning before a failure

about 4 hours

271 of 271 test failures caught, 95% of alerts real

① See it coming, fix it in a break

Bjørn's app, live on the Aurora Elbil line

② Know your exposure

Which OEM contract is at risk right now, in plain language

③ Run it safely in production

How it is built, deployed, secured and owned

CHECK IN "Alex, Shreya: does this match what you want to get out of today? Anything you'd add?"

① See it coming, fix it in a planned break: Tell

3:00 to 4:00 · before the demo · for both

> TODAY ON THE SHOP FLOOR Bjørn finds out when the line stops. His crew diagnoses on the spot while the JIS clock runs. Preventive maintenance follows a calendar, so it is too early for some stations and too late for others.

What I'll highlight: one station, PLT-E-A03, the frame welder on the Aurora Elbil line (60-minute buffer). In the demo, one minute is about one operating hour.

Step	What you'll see
1	The welder's risk climbs from green to **HIGH**
2	The screen tells Bjørn **why**: which sensor, and how far off normal
3	**One click** creates the work order for a planned break
4	The repair is done, and the risk returns to **NORMAL**

① See it coming, fix it in a planned break: Show, then Tell

4:00 to 8:30 · the demo · for both

SHOW ▶ **Plant Health Live app**

- **Step 1 to 2:** the welder goes from about 1% to HIGH (79%, then 99%). The top signal is motor current, 10% above normal.
- **Step 3:** one click creates a P1 work order. The screen answers in milliseconds, so Bjørn decides fast, with no waiting and no re-typing.
- **Step 4:** the technician completes it, and the risk returns to NORMAL on the same screen.

SO WHAT Bjørn's team gets **hours of warning instead of an emergency stop**: a 28-minute planned job instead of 112 minutes, about $6,300 less in parts, and no $575,000 Aurora line stop.

ASK SHREYA "Shreya, if your team owned this app, who would be the first call when Bjørn says the screen looks wrong? Let's make sure that's covered in topic 3."

② Know your exposure

8:30 to 11:30 · Tell, Show, Tell · for Alex

TELL "Today, before an OEM review, someone builds a spreadsheet of which lines are at risk. It's out of date when it's printed. I'll ask one question in plain language and get the answer from live data."

SHOW ▶ **Genie: Plant Maintenance Agent : "Which OEM programs are exposed to a line stop right now?"**

Line	OEM program	Riskiest station	OEM stop if it fails	Risk-weighted exposure
PLT-S-B	Fjord Electric FE Pace Sedan	Stamping press, HIGH	51 min	**$913k**
PLT-N-C	Nordvik Motors NV-e5 Crossover	Stamping press, HIGH	51 min	**$609k**
PLT-E-A	Aurora Elbil A3 Hatch	Extrusion CNC, ELEVATED	55 min	**$294k**

Earlier snapshot; the live answer reflects today's risk. The answer comes from one governed view (**pdm_ops.oem_delivery_exposure**). Genie answered 12 of 12 test questions correctly before anyone used it.

SO WHAT **You walk into the Fjord Electric review knowing your exposure**, and Bjørn's crew works the riskiest line first. Fewer stops, fewer charges.

ASK ALEX "Alex, which OEM relationship worries you most right now? That's the line I'd pick for the pilot."

③ Run it safely in production: how it works

11:30 to 14:00 · Tell, Show, Tell · for Shreya

> TELL "Shreya, your questions are who runs it, what breaks, and how it's secured. First the flow, one benefit per step. Then the path to production."

Plant gateways
push sensor data directly, no message broker to run

→

Pipeline
cleans every reading and scores every station every 10 s

→

Operational database
serves Bjørn's screen in milliseconds and stores work orders

→

App
one screen for the crew, one question box for leadership

Sensor data reaches Bjørn's screen in under a minute. Failures build over hours, so the decision window is the next planned break.

> SHOW ▶ **pipeline (data-quality tab)** · **operational database (Lakebase)** · **model endpoint + AI Gateway**

> SO WHAT Every part is a managed service: **no brokers, database servers or model hosts for Shreya's team to patch**. Alex: that keeps run cost and on-call small.

③ Run it safely in production: code to prod, security, ownership

14:00 to 17:00 · for Shreya, answer this even if she doesn't ask

From code to production

- **Deploy:** pipeline, jobs (including the Zerobus target setup), app and model endpoint are one bundle in Git; the Lakebase synced table is one CLI step today (**databricks.yml**)
- **Promote:** CI/CD validates the bundle, deploys to test, runs an end-to-end test and a data-quality gate, then promotes to prod after approval
- **Proven:** **mfg-telemetry-dq-cicd** runs dev → staging with the gate. A bad build failed it, so it never reached prod
- **Roll back:** redeploy the previous bundle version, or move the model's `@champion` alias back
- **Bad data and drift:** expectations drop and count bad readings; Zerobus rejects records that don't match the table; for file sources, quarantine keeps bad rows and schema changes are rescued, not crashed

Security and integration

- **Access:** Unity Catalog grants per identity; plant row filters, proven on the app's identity (6 → 12 → 6 rows)
- **PII:** technician contact data masked
- **Secrets:** in secret scopes, never in code; the app's credentials are injected by the platform
- **Identity:** SSO and SCIM from your identity provider
- **Network:** private connectivity from plant gateways to the workspace, designed with your network team in the pilot
- **CMMS:** work orders already flow into the lakehouse; the pilot adds the push into your CMMS

On-call: pipeline failures and an hourly data-quality monitor alert by email, or to Teams through a notification destination. Your team owns the app code; the platform runs the infrastructure.

ASK SHREYA "Shreya, does this fit how your team ships today, or is there a step in your release process we should plug into?"

How sure can you be? Proof, and the honest limits

17:00 to 18:30 · for both · every number is an executed run (**requirement → evidence matrix**)

IF ASKED ABOUT SYNTHETIC DATA, ANSWER NOW "Fair question. The model was tested on a later time window it never saw. In the pilot it retrains on your history, we validate the failure patterns with your engineers, and shadow mode is the go/no-go gate before any alert goes live."

Proven on the running system

- 271 of 271 failures flagged, about 4 operating hours ahead, 95% precision
- The live loop: fault → HIGH → work order → repair → NORMAL; the station never failed
- 96 stations at 1 Hz; reads in 4 to 32 ms; work orders in 12 ms
- Genie 12 of 12; plant isolation proven

Honest limits

- Synthetic data and fictional contracts: the value is illustrative until the pilot
- About 60 s to screen: right for this decision, not a safety interlock
- Alerts name the most deviating sensor, not a full root cause

SO WHAT You don't have to trust our numbers. **The pilot proves them on your line, behind a shadow-mode gate.**

Pilot and the ask

18:30 to 20:00 · for both

90 days on one line, PLT-E-A (Aurora Elbil A3 Hatch, Plant East)

Weeks	What happens
1 to 2	Baseline: 12 months of failures, repair times, downtime cost, OEM charges
3 to 5	Connect the line's sensors; load CMMS history and JIS terms; Shreya's team deploys the bundle into your workspace
6 to 8	Train on your history; validate the patterns with your engineers; set the alert threshold with Bjørn
9 to 12	**Shadow mode** first, as the go/no-go gate, then live alerts

Success, agreed up front: at least 70% of failures flagged with more than 60 minutes of warning · at least 80% alert precision · zero OEM stops from flagged stations · a measured drop in unplanned downtime

THE ASK **Alex:** sponsor the 90-day pilot on PLT-E-A. **Shreya:** a code walkthrough and the first deploy into your workspace next week.

Backup · demo launchpad (every component, live)

Links open in a new tab. Workspace: **fevm-serverless-stable-am1uc2**.

Step	Open	What to show
Data generation + **Zerobus**	**simulator job** · **pdm_raw.sensor_readings**	Job running, 96 stations at 1 Hz. Table history: every write `engineInfo = Zerobus`
SDP pipeline	**pipeline** · **pdm_core.sensor_readings_clean** · **pdm_core.station_risk_scores** · **pdm_core.station_health_current**	Graph, data-quality tab, lineage from the Zerobus table to the app
ML in Unity Catalog	**UC model** · **MLflow experiment** · **training job**	Version 2 = `@champion`, PR-AUC 0.91, lead time per failure mode
Lakebase	**synced-table pipeline** · **pdm_live.station_risk_scores** · **operational database (Lakebase)**	Synced table about 4 s behind Delta; work orders as Postgres rows
App	**Plant Health Live app** · **app config**	Inject fault → HIGH → work order → NORMAL
Model Serving + AI Gateway	**model endpoint + AI Gateway**	What-if from the app, usage tracking
Genie	**Genie: Plant Maintenance Agent**	OEM exposure question; benchmarks 12/12
Governed KPIs	**pdm_ops.maintenance_metrics** · **pdm_ops.station_risk_metrics** · **pdm_ops.oem_delivery_exposure**	One definition per KPI; OEM exposure joins live risk to JIS contracts
Governance	**pdm_raw.technicians** · **pdm_ops.work_orders_current** · **governance job**	PII masks, plant row filter, tags, grants
Industry context	**industry context job**	OEM programs, JIS buffers, scorecard KPIs computed from history
Code and proof	**GitHub repo** · **databricks.yml** · **src/pipeline** · **src/app** · **evidence/** · **evidence notebooks job**	One bundle; executed notebooks with outputs
CI/CD and data quality	**mfg-telemetry-dq-cicd**	Zerobus + Auto Loader, SQL expectations, quarantine, DQ gate, dev → staging → prod

Before demoing: start Lakebase, both pipelines, the simulator and the app; warm the serving endpoint and Genie once.

Backup · the value in detail

How an executive sponsor at a Tier-1 supplier is judged (Volta to confirm)	What moves
EBITDA margin after annual OEM price-downs	about $3.6M a year from less downtime and fewer expedited parts, no extra headcount
OEM charges and premium freight	about $4.0M a year if 5% of failures reach the OEM (2%: $1.6M; 10%: $8.0M)
OEM scorecard and the next program award	fewer line stops; exposure per OEM visible live
Capex and free cash flow	about +0.26 OEE availability points from the same 12 lines

Bjørn's crew	Today	With 70% of failures caught
Planned share of repairs	20%	about 76%
Repair minutes per job	96	about 48
Alerts they can trust	none	95% precision

Assumptions: 2 unplanned failures per station per year, 70% converted into planned repairs, $15,000 per hour of internal line downtime, $595k average OEM charge per stop.

Backup · where the numbers come from

All figures are from executed runs, committed as text in **GitHub repo**. How the industry review gaps were closed: `docs/INDUSTRY_FIT.md`.

Claim	Evidence
271/271 failures flagged, median 244 s lead (about 4 operating hours), PR-AUC 0.91, precision 0.95	`evidence/04_training/02_train_model.md`
112 vs 28 min downtime, $7,200 vs $870 parts, 1,096 corrective vs 268 preventive repairs	`evidence/03_backfill/01_backfill_history.md`
OEM programs, JIS buffers, line-stop charges, exposure per line and per OEM, $595k average charge	`evidence/11_industry_context/05_industry_context.md`
Planned share, repair minutes, parts spend, layer 1 and 2 value, OEE points, sensitivity	`evidence/11_industry_context/05_industry_context.md`, "Scorecard KPIs", "Value model", "Sensitivity"
Live run on PLT-E-A03: fault, HIGH, work order, repair, NORMAL	`evidence/08_app/e2e_fault_injection.md`, `evidence/10_evidence_notebooks/E08_end_to_end.md`
3.9 s to clean data, about 60 s to screen, 4 to 32 ms reads, 12 ms work orders	`evidence/06_live_pipeline/04_live_evidence.md`, `evidence/08_app/`, `evidence/10_evidence_notebooks/E04_lakebase.md`
Genie 12/12, including the OEM exposure questions	`evidence/07_genie/benchmark.md`, `evidence/10_evidence_notebooks/E06_genie.md`
Plant-level access on the app identity (6 → 12 → 6 rows)	`evidence/08_app/governance_as_app_sp.json`, `evidence/10_evidence_notebooks/E05_governance.md`
Promotion dev → staging with an integration test and DQ gate; a bad build blocked	**mfg-telemetry-dq-cicd**, `evidence/`

Layer 1: 96 × 2 = 192 failures; 70% = 134 converted; 134 × 84.6 min = 189 h × $15k = $2.83M; 134 × $6,329 = $0.85M; false alarms ≈ 7 × $7.9k = $0.06M; net $3.62M. **Layer 2:** 192 × 5% = 9.6 OEM stops × 70% × $594,853 = $4.0M. **Maintenance KPIs:** 329 corrective + 1,035 planned: planned share 76%, repair minutes 48, parts $8.12M → $3.27M.

Volta Industrial, Nordvik Motors, Fjord Electric, Aurora Elbil and all personas are fictional, and so is all of the data.